

DOCUMENTAÇÃO TÉCNICA DO SISTEMA

CineMax Locadora

Gerenciador Operacional de Mídias

Unidade curricular	Programação Orientada a Objetos (POO)
Curso	Técnico em Informática — Escola Técnica Albert Einstein (SEG)
Estudante/Equipe	João Victor Fagundes de Paula · Guilherme Carvalho · Cristofer · Arthur Franco de Andrade
Docente	Lucas Camponogara Machado
Versão do sistema	1.0
Data	30/09/2026 (apresentação do projeto: 03/10/2026)

1. Identificação técnica do projeto

Nome do sistema	CineMax Locadora — Gerenciador Operacional de Mídias
Versão	1.0
Linguagem	PHP 8.x, com Programação Orientada a Objetos
Ambiente de execução	Visual Studio Code e XAMPP (Apache), ou linha de comando com <code>php index.php</code>
Repositório GitHub	https://github.com/Vyctorr7/Locadora
Data da versão	30/09/2026

2. Visão técnica da solução

O sistema foi desenvolvido em **PHP** seguindo o paradigma da **Programação Orientada a Objetos**. Ele representa uma rede de locadoras de mídias físicas (DVD e Blu-Ray) e resolve três problemas operacionais: o cálculo de preços diferentes conforme o tipo e o estado da mídia, o controle de disponibilidade do acervo e a cobrança de multa por devolução atrasada.

A solução é composta por **8 classes**, organizadas em quatro módulos. O módulo de **Pessoas** (Pessoa, Cliente, Funcionario) guarda os dados de identificação e o saldo devedor. O módulo de **Mídias** (Midia, DVD, BluRay) guarda o acervo e define a regra de preço de cada formato. O módulo de **Finanças** (Locacao) une cliente, funcionário e mídias em um contrato e calcula o valor e as multas. O módulo **Central** (Locadora) mantém as coleções de clientes, funcionários, mídias e locações.

Os objetos colaboram da seguinte forma: a Locadora armazena os cadastros; uma Locacao recebe um Cliente, um Funcionario e uma lista de Midia; ao calcular o total, a Locacao chama `calcularPrecoLocacao()` em cada mídia, e cada subclasse aplica a própria regra. O **ponto de entrada** é o arquivo `index.php`, que instancia os objetos e executa a simulação completa de uma locação, com devolução atrasada.

Divisão de responsabilidades da equipe

Integrante	Módulo	Arquivos
João Victor Fagundes de Paula	Mídias e polimorfismo	Midia.php, DVD.php, BluRay.php
Guilherme Carvalho	Pessoas	Pessoa.php, Cliente.php, Funcionario.php
Cristofer	Central e sistema	Locadora.php
Arthur Franco de Andrade	Finanças e testes	Locacao.php, index.php

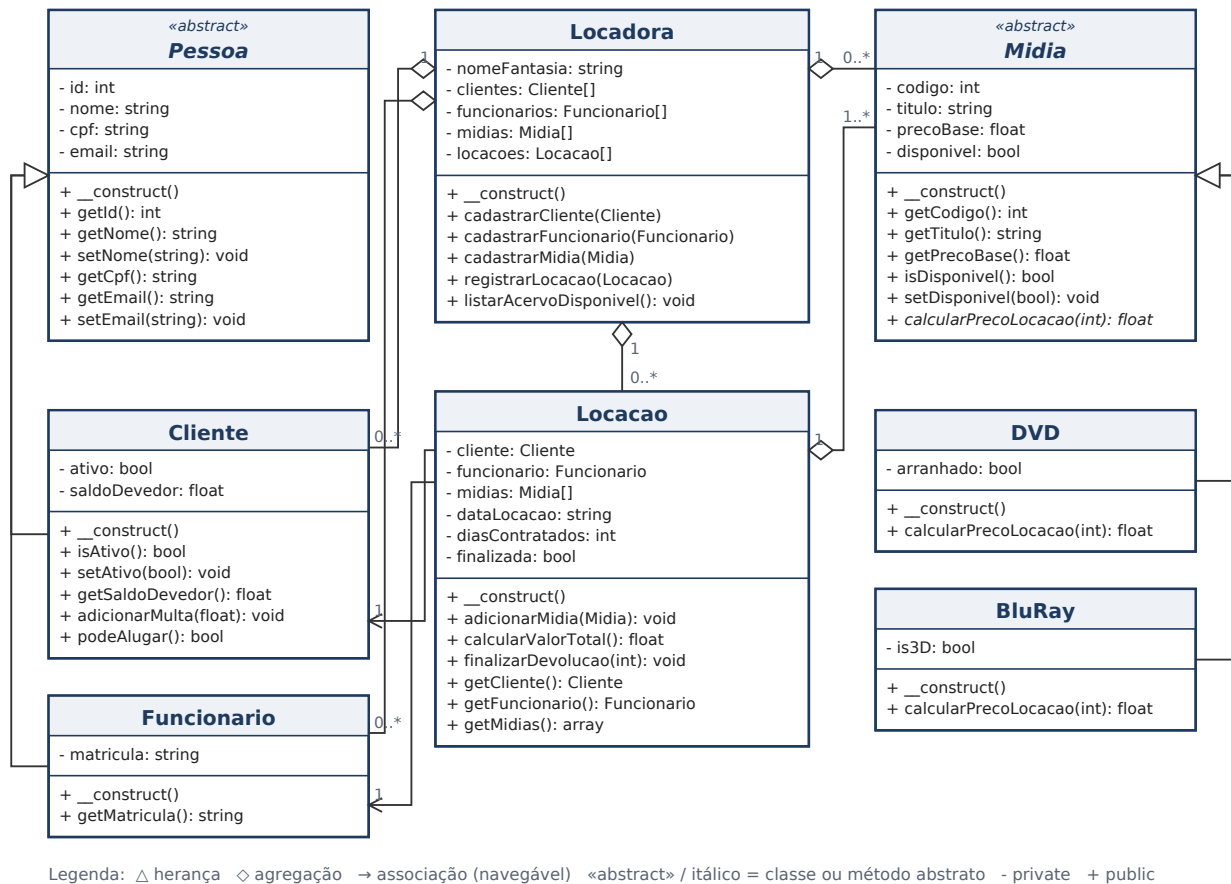
3. Estrutura de diretórios e arquivos

```
ProjetoEquipe/  
├── Classes/  
│   ├── Pessoa.php  
│   ├── Cliente.php  
│   ├── Funcionario.php  
│   ├── Midia.php  
│   ├── DVD.php  
│   ├── BluRay.php  
│   ├── Locacao.php  
│   └── Locadora.php  
├── DOCS/  
│   └── Documentacao_Tecnica_CineMax.pdf  
├── .gitignore  
├── index.php  
├── link_repositorio.txt  
└── README.md
```

Diretório/arquivo	Responsabilidade técnica
Classes/Pessoa.php	Superclasse abstrata com os dados de identificação de uma pessoa.
Classes/Cliente.php	Subclasse de Pessoa; controla status ativo e saldo devedor e decide se o cliente pode alugar.
Classes/Funcionario.php	Subclasse de Pessoa; acrescenta a matrícula funcional.
Classes/Midia.php	Superclasse abstrata do acervo; declara o método abstrato <code>calcularPrecoLocacao()</code> .
Classes/DVD.php	Subclasse de Midia; aplica desconto de 10% quando o disco está arranhado.
Classes/BluRay.php	Subclasse de Midia; acrescenta taxa fixa de R\$ 5,00 quando é 3D.
Classes/Locacao.php	Contrato de locação: associa cliente e funcionário, agrega mídias, calcula o total e processa a devolução com multa.
Classes/Locadora.php	Classe gestora: mantém as coleções do sistema e lista o acervo disponível.
index.php	Ponto de entrada; carrega as classes e executa a simulação do fluxo completo.
DOCS/	Documentação técnica em PDF.
README.md	Apresentação do projeto no GitHub.
.gitignore	Regras de versionamento; ignora <code>link_repositorio.txt</code> .
link_repositorio.txt	Endereço do repositório, entregue no ZIP e não versionado.

4. Diagrama de classes

O diagrama mostra as oito classes com atributos, métodos, visibilidade, herança, relacionamentos e multiplicidades, conforme a versão final do código.



Decisões apresentadas no diagrama

- Existem duas hierarquias de herança: Pessoa $\rightarrow$ Cliente/Funcionario e Midia $\rightarrow$ DVD/BluRay. Nas duas, a superclasse é **abstrata**, pois não faz sentido existir uma “pessoa” ou uma “mídia” genérica.
- Todos os atributos são private. As subclasses acessam os dados da superclasse pelos getters públicos, por exemplo `getPrecoBase()` dentro de DVD.
- Locacao tem **associação** com Cliente e Funcionario e **agregação** com Midia: as mídias continuam existindo no acervo depois que o contrato termina.
- Locadora agrega as coleções de clientes, funcionários, mídias e locações, recebendo objetos já criados.
- O método abstrato `calcularPrecoLocacao()` permite que Locacao calcule o total sem conhecer o tipo de cada mídia.

5. Dicionário de classes

5.1 Classe: Pessoa

Arquivo	Classes/Pessoa.php
Responsabilidade	Superclasse abstrata que reúne os dados de identificação comuns a qualquer pessoa do sistema.
Superclasse (se houver)	Nenhuma (raiz da hierarquia de pessoas). Classe abstract.
Classes relacionadas	Cliente, Funcionario (subclasses).
Observações	Não pode ser instanciada diretamente. id e cpf não possuem setter, portanto não mudam depois da criação.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$id	int	private	Identificador da pessoa.
\$nome	string	private	Nome completo.
\$cpf	string	private	Documento de identificação.
\$email	string	private	E-mail de contato.

Métodos (todos com visibilidade public)

Método	Parâmetros	Retorno	Finalidade
__construct()	int \$id, string \$nome, string \$cpf, string \$email	—	Inicializa os quatro atributos.
getId()	—	int	Retorna o identificador.
getNome()	—	string	Retorna o nome.
setNome()	string \$nome	void	Altera o nome.
getCpf()	—	string	Retorna o CPF.
getEmail()	—	string	Retorna o e-mail.
setEmail()	string \$email	void	Altera o e-mail.

5.2 Classe: Cliente

Arquivo	Classes/Cliente.php
Responsabilidade	Representa quem aluga mídias; guarda o status no sistema e as multas pendentes.
Superclasse (se houver)	Pessoa
Classes relacionadas	Pessoa, Locacao, Locadora.
Observações	Nasce ativo e com saldo devedor zero. A regra de quem pode alugar está no método podeAlugar().

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$ativo	bool	private	Indica se o cliente está ativo.
\$saldoDevedor	float	private	Soma das multas pendentes.

Métodos (todos com visibilidade public)

Método	Parâmetros	Retorno	Finalidade
__construct()	int \$id, string \$nome, string \$cpf, string \$email	—	Chama parent::__construct() e define ativo = true e saldoDevedor = 0.0.
isAtivo()	—	bool	Retorna o status.
setAtivo()	bool \$ativo	void	Ativa ou desativa o cliente.
getSaldoDevedor()	—	float	Retorna o saldo devedor.
adicionarMulta()	float \$valor	void	Soma o valor ao saldo devedor.
podeAlugar()	—	bool	Retorna verdadeiro se o cliente está ativo e sem saldo devedor.

5.3 Classe: Funcionario

Arquivo	Classes/Funcionario.php
Responsabilidade	Representa o atendente responsável por uma locação.
Superclasse (se houver)	Pessoa
Classes relacionadas	Pessoa, Locacao, Locadora.
Observações	A matrícula não possui setter.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$matricula	string	private	Código de registro na empresa.

Métodos (todos com visibilidade public)

Método	Parâmetros	Retorno	Finalidade
__construct()	int \$id, string \$nome, string \$cpf, string \$email, string \$matricula	—	Chama parent::__construct() e define a matrícula.
getMatricula()	—	string	Retorna a matrícula.

5.4 Classe: Midia

Arquivo	Classes/Midia.php
Responsabilidade	Superclasse abstrata do acervo; guarda código, título, preço base e disponibilidade.
Superclasse (se houver)	Nenhuma (raiz da hierarquia de mídias). Classe abstract.
Classes relacionadas	DVD, BluRay (subclasses), Locacao, Locadora.
Observações	Declara calcularPrecoLocacao() como método abstrato: cada subclasse é obrigada a implementá-lo. A mídia nasce disponível.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$codigo	int	private	Identificador da mídia.
\$titulo	string	private	Título do filme.
\$precoBase	float	private	Valor da diária.
\$disponivel	bool	private	Indica se pode ser alugada.

Métodos (todos com visibilidade public)

Método	Parâmetros	Retorno	Finalidade
__construct()	int \$codigo, string \$titulo, float \$precoBase	—	Inicializa os atributos e define disponivel = true.
getCodigo()	—	int	Retorna o código.
getTitulo()	—	string	Retorna o título.
getPrecoBase()	—	float	Retorna o preço base.
isDisponivel()	—	bool	Retorna a disponibilidade.
setDisponivel()	bool \$disponivel	void	Atualiza a disponibilidade.
calcularPrecoLocacao()	int \$dias	float	Abstrato. Calcula o valor da locação da mídia.

5.5 Classe: DVD

Arquivo	Classes/DVD.php
Responsabilidade	Mídia do tipo DVD; aplica desconto quando o disco possui arranhões.
Superclasse (se houver)	Midia
Classes relacionadas	Midia, Locacao, Locadora.
Observações	Acessa o preço base pelo getter getPrecoBase(), pois o atributo da superclasse é private.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$arranhado	bool	private	Indica se o disco está arranhado.

Métodos (todos com visibilidade public)

Método	Parâmetros	Retorno	Finalidade
__construct()	int \$codigo, string \$titulo, float \$precoBase, bool \$arranhado = false	—	Chama parent::__construct() e define o estado físico.
calcularPrecoLocacao()	int \$dias	float	Retorna precoBase × dias; se arranhado, multiplica por 0,90.

5.6 Classe: BluRay

Arquivo	Classes/BluRay.php
Responsabilidade	Mídia do tipo Blu-Ray; aplica taxa fixa quando é 3D.
Superclasse (se houver)	Midia
Classes relacionadas	Midia, Locacao, Locadora.
Observações	A taxa de R\$ 5,00 é somada uma vez ao total, e não por dia.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$is3D	bool	private	Indica se o Blu-Ray é 3D.

Métodos (todos com visibilidade public)

Método	Parâmetros	Retorno	Finalidade
__construct()	int \$codigo, string \$titulo, float \$precoBase, bool \$is3D = false	—	Chama parent::__construct() e define o suporte a 3D.
calcularPrecoLocacao()	int \$dias	float	Retorna precoBase × dias; se 3D, soma R\$ 5,00.

5.7 Classe: Locacao

Arquivo	Classes/Locacao.php
Responsabilidade	Contrato de locação: associa cliente e funcionário, agrega as mídias alugadas, calcula o total e processa a devolução.
Superclasse (se houver)	Nenhuma.
Classes relacionadas	Cliente, Funcionario, Midia (e subclasses), Locadora.
Observações	dataLocacao é gravada no construtor, mas não é usada nos cálculos desta versão. finalizada não possui getter.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$cliente	Cliente	private	Cliente da locação.
\$funcionario	Funcionario	private	Funcionário que atendeu.
\$midias	array (Midia[])	private	Mídias incluídas no contrato.
\$dataLocacao	string	private	Data de criação, formato Y-m-d.
\$diasContratados	int	private	Quantidade de dias do aluguel.
\$finalizada	bool	private	Indica se a devolução já foi processada.

Métodos (todos com visibilidade public)

Método	Parâmetros	Retorno	Finalidade
__construct()	Cliente \$cliente, Funcionario \$funcionario, int \$diasContratados	—	Inicializa o contrato aberto, com lista de mídias vazia.
adicionarMidia()	Midia \$midia	void	Inclui a mídia se estiver disponível e a marca como alugada.
calcularValorTotal()	—	float	Soma o preço de cada mídia usando polimorfismo.
finalizarDevolucao()	int \$diasEfetivosAtraso = 0	void	Libera as mídias, aplica multa se houver atraso e encerra o contrato.
getCliente()	—	Cliente	Retorna o cliente.
getFuncionario()	—	Funcionario	Retorna o funcionário.
getMidias()	—	array	Retorna as mídias do contrato.

5.8 Classe: Locadora

Arquivo	Classes/Locadora.php
Responsabilidade	Classe gestora: mantém as coleções de clientes, funcionários, mídias e locações e exibe o acervo disponível.
Superclasse (se houver)	Nenhuma.
Classes relacionadas	Cliente, Funcionario, Midia, Locacao.
Observações	Os dados ficam apenas em memória, em arrays, durante a execução.

Atributos

Atributo	Tipo	Visibilidade	Finalidade
\$nomeFantasia	string	private	Nome da locadora.
\$clientes	array (Cliente[])	private	Clientes cadastrados.
\$funcionarios	array (Funcionario[])	private	Funcionários cadastrados.
\$midias	array (Midia[])	private	Acervo.
\$locacoes	array (Locacao[])	private	Loações registradas.

Métodos (todos com visibilidade public)

Método	Parâmetros	Retorno	Finalidade
__construct()	string \$nomeFantasia	—	Define o nome e inicia as coleções vazias.
cadaststrarCliente()	Cliente \$cliente	void	Adiciona o cliente à coleção.
cadaststrarFuncionario()	Funcionario \$funcionario	void	Adiciona o funcionário à coleção.
cadaststrarMidia()	Midia \$midia	void	Adiciona a mídia ao acervo.
registrarLocacao()	Locacao \$locacao	void	Registra o contrato de locação.
listarAcervoDisponivel()	—	void	Imprime as mídias disponíveis com tipo, título e preço base.

6. Especificação dos métodos principais

São documentados os métodos que concentram as regras de negócio. Getters e setters simples não foram repetidos.

6.1 Método: Locacao::adicionarMidia()

Assinatura	public function adicionarMidia(Midia \$midia): void
Responsabilidade	Incluir a mídia no contrato somente se ela estiver disponível e marcá-la como alugada.
Parâmetros/entrada	Um objeto Midia (DVD ou BluRay).
Retorno	void. Se a mídia já estiver alugada, imprime a mensagem "A mídia '<título>' já está alugada!".
Estado alterado	Mídia disponível: entra no array midias da locação e recebe disponivel = false. Mídia indisponível: nada muda.
Classes/objetos envolvidos	Locacao, Midia (DVD/BluRay).
Observações	Impede que o mesmo item seja locado duas vezes.

6.2 Método: Locacao::calcularValorTotal()

Assinatura	public function calcularValorTotal(): float
Responsabilidade	Somar o valor de todas as mídias do contrato para os dias contratados.
Parâmetros/entrada	Nenhum; usa diasContratados e o array midias.
Retorno	float com a soma de calcularPrecoLocacao(\$diasContratados) de cada mídia.
Estado alterado	Nenhum (apenas consulta).
Classes/objetos envolvidos	Locacao, DVD, BluRay.
Observações	Ponto de polimorfismo : a chamada é a mesma para qualquer mídia e o resultado depende da subclasse.

6.3 Método: Locacao::finalizarDevolucao()

Assinatura	public function finalizarDevolucao(int \$diasEfetivosAtraso = 0): void
Responsabilidade	Encerrar a locação, liberar as mídias e cobrar multa em caso de atraso.
Parâmetros/entrada	Número de dias de atraso (padrão 0), informado por quem chama o método.
Retorno	void.
Estado alterado	Cada mídia volta a disponivel = true; se dias > 0, o saldoDevedor do cliente aumenta em dias × 5,00; finalizada = true.
Classes/objetos envolvidos	Locacao, Midia, Cliente.
Observações	Os dias de atraso não são calculados a partir de datas nesta versão.

6.4 Método: Cliente::podeAlugar()

Assinatura	<code>public function podeAlugar(): bool</code>
Responsabilidade	Decidir se o cliente está apto a fazer uma nova locação.
Parâmetros/entrada	Nenhum; usa ativo e saldoDevedor.
Retorno	true se ativo for verdadeiro e saldoDevedor == 0.0; caso contrário, false.
Estado alterado	Nenhum.
Classes/objetos envolvidos	Cliente.
Observações	É consultado pelo <code>index.php</code> antes de criar a locação.

6.5 Método: DVD::calcularPrecoLocacao()

Assinatura	<code>public function calcularPrecoLocacao(int \$dias): float</code>
Responsabilidade	Calcular o valor da locação de um DVD.
Parâmetros/entrada	Quantidade de dias.
Retorno	<code>precoBase × dias</code> ; se arranhado, o total é multiplicado por 0,90 (desconto de 10%).
Estado alterado	Nenhum.
Classes/objetos envolvidos	DVD, Midia.
Observações	Exemplo do sistema: R\$ 5,00 × 3 dias = R\$ 15,00; arranhado → R\$ 13,50.

6.6 Método: BluRay::calcularPrecoLocacao()

Assinatura	<code>public function calcularPrecoLocacao(int \$dias): float</code>
Responsabilidade	Calcular o valor da locação de um Blu-Ray.
Parâmetros/entrada	Quantidade de dias.
Retorno	<code>precoBase × dias</code> ; se is3D, soma R\$ 5,00 ao total.
Estado alterado	Nenhum.
Classes/objetos envolvidos	BluRay, Midia.
Observações	Exemplo do sistema: R\$ 8,00 × 3 dias + R\$ 5,00 = R\$ 29,00.

6.7 Método: Locadora::listarAcervoDisponivel()

Assinatura	<code>public function listarAcervoDisponivel(): void</code>
Responsabilidade	Mostrar em tela as mídias que podem ser alugadas.
Parâmetros/entrada	Nenhum.
Retorno	void. Imprime um cabeçalho e uma linha por mídia disponível: - [Classe] Título (R\$ preço/dia).
Estado alterado	Nenhum.
Classes/objetos envolvidos	Locadora, Midia.
Observações	Usa <code>get_class()</code> para mostrar o tipo real do objeto (DVD ou BluRay).

7. Relacionamentos entre classes

Os relacionamentos abaixo foram conferidos no código-fonte. Não há relação de **composição** nesta versão: nenhuma classe cria ou destrói os objetos das outras, pois `Cliente`, `Funcionario` e `Midia` são criados fora e apenas passados por parâmetro.

Classe A	Classe B	Tipo de relação	Como aparece no código	Justificativa
Locacao	Cliente	Associação (N para 1)	<code>private Cliente \$cliente,</code> <code>recebido no construtor.</code>	Toda locação pertence a um cliente, que continua existindo depois dela.
Locacao	Funcionario	Associação (N para 1)	<code>private Funcionario \$funcionario,</code> <code>recebido no construtor.</code>	Registra quem atendeu, garantindo rastreabilidade.
Locacao	Midia	Agregação (1 para 1..*)	<code>private array \$midias;</code> <code>adicionarMidia(Midia \$midia).</code>	As mídias são do acervo e sobrevivem ao fim do contrato; a devolução só as libera.
Locadora	Cliente	Agregação (1 para 0..*)	<code>private array \$clientes;</code> <code>cadastrarCliente().</code>	A locadora guarda objetos criados fora dela.
Locadora	Funcionario	Agregação (1 para 0..*)	<code>private array \$funcionarios;</code> <code>cadastrarFuncionario().</code>	Mesmo caso dos clientes.
Locadora	Midia	Agregação (1 para 0..*)	<code>private array \$midias;</code> <code>cadastrarMidia().</code>	Forma o acervo da empresa.
Locadora	Locacao	Agregação (1 para 0..*)	<code>private array \$locacoes;</code> <code>registrarLocacao().</code>	Mantém o histórico de contratos registrados.
Cliente	Pessoa	Herança	<code>class Cliente extends Pessoa</code>	Cliente é uma pessoa com dados extras.
Funcionario	Pessoa	Herança	<code>class Funcionario extends Pessoa</code>	Funcionário é uma pessoa com matrícula.
DVD	Midia	Herança	<code>class DVD extends Midia</code>	DVD é um tipo de mídia com regra de preço própria.
BluRay	Midia	Herança	<code>class BluRay extends Midia</code>	Blu-Ray é um tipo de mídia com regra de preço própria.

8. Herança, generalização e especialização

O sistema possui **duas hierarquias**, ambas com superclasse abstrata.

Hierarquia de pessoas

- **Superclasse:** `Pessoa` (generalização), com `id`, `nome`, `cpf` e `email`.
- **Subclasses:** `Cliente` (acrescenta `ativo` e `saldoDevedor`) e `Funcionario` (acrescenta `matricula`).
- **Elementos herdados:** getters e setters de `Pessoa`, por exemplo `getNome()`, usado em `Cliente` e `Funcionario`.
- **Por que faz sentido:** clientes e funcionários são pessoas com os mesmos dados básicos; a herança evita repetir esse código.

Hierarquia de mídias

- **Superclasse:** `Midia` (generalização), com `codigo`, `titulo`, `precoBase` e `disponivel`, e o método abstrato `calcularPrecoLocacao()`.

- **Subclasses:** DVD (acrescenta arranhado) e BluRay (acrescenta is3D).
- **Comportamento especializado:** cada subclasse implementa o cálculo do preço à sua maneira.
- **Por que faz sentido:** o controle de disponibilidade e os dados básicos são iguais para todas as mídias; só a regra de preço muda. Um novo formato pode ser criado sem alterar Locacao.

Como os atributos são `private`, as subclasses usam os getters das superclasses, por exemplo `$this->getPrecoBase()` em DVD e BluRay, e `parent::__construct()` para inicializar a parte herdada.

9. Sobrescrita e polimorfismo

Elemento	Classe(s) envolvida(s)	Método/comportamento	Explicação técnica
Implementação de método abstrato (sobrescrita)	Midia, DVD, BluRay	<code>calcularPrecoLocacao(int \$dias)</code>	Midia declara o método sem corpo; DVD e BluRay o implementam com regras diferentes (desconto de 10% e taxa de R\$ 5,00).
Polimorfismo de subtipo	Locacao, Midia, DVD, BluRay	<code>calcularValorTotal()</code>	O <code>foreach</code> percorre o array <code>midias</code> e chama <code>calcularPrecoLocacao()</code> sem verificar o tipo; o PHP executa a versão da subclasse de cada objeto.
Polimorfismo na listagem	Locadora, Midia	<code>listarAcervoDisponivel()</code>	Os itens são tratados como Midia (<code>isDisponivel()</code> , <code>getTitulo()</code> , <code>getPrecoBase()</code>), e <code>get_class()</code> mostra o tipo real.
Sobrescrita de construtor	Cliente, Funcionario, DVD, BluRay	<code>__construct()</code>	Cada subclasse declara seu construtor com parâmetros extras e chama <code>parent::__construct()</code> para inicializar a superclasse.

Chamada polimórfica (Locacao::calcularValorTotal)

```
foreach ($this->midias as $midia) {
    $total += $midia->calcularPrecoLocacao($this->diasContratados);
}
```

Com 3 dias contratados, o resultado no `index.php` é: DVD comum = R\$ 15,00; DVD arranhado = R\$ 13,50; Blu-Ray 3D = R\$ 29,00. O total é **R\$ 57,50**.

10. Fluxos técnicos do sistema

10.1 Fluxo: locação de mídias e cálculo do valor

```
index.php
  ↓ instancia e cadastra
Locadora · Cliente · Funcionario · DVD (2) · BluRay (1)
  ↓ consulta
Cliente::podeAlugar() → true (ativo e saldoDevedor = 0)
  ↓ cria
Locacao($cliente, $funcionario, 3 dias)
  ↓ chama 3 vezes
Locacao::adicionarMidia($midia) → Midia::setDisponivel(false)
  ↓ registra
Locadora::registrarLocacao($locacao)
  ↓ chama
Locacao::calcularValorTotal()
  ↓ para cada mídia (polimorfismo)
DVD → 15,00 | DVD arranhado → 13,50 | BluRay 3D → 29,00
  ↓
Total = R$ 57,50
```

O index.php verifica se o cliente pode alugar, cria a locação, adiciona as três mídias (que passam a indisponíveis) e registra o contrato na Locadora. Em seguida, calcularValorTotal() soma o preço de cada mídia, e cada subclasse aplica sua regra.

10.2 Fluxo: devolução com atraso e multa

```
index.php
  ↓ chama
Locacao::finalizarDevolucao(2)
  ↓ para cada mídia
Midia::setDisponivel(true)
  ↓ atraso > 0: multa = 2 × 5,00 = 10,00
Cliente::adicionarMulta(10.0) → saldoDevedor = 10,00
  ↓
finalizada = true
  ↓ nova consulta
Cliente::podeAlugar() → false (multa pendente)
```

A devolução libera as mídias e transforma os dias de atraso em multa no saldo do cliente. Enquanto houver saldo devedor, podeAlugar() retorna false, e as mídias voltam a aparecer no acervo disponível.

11. Regras e validações implementadas

ID	Regra/validação	Classe responsável	Método	Tratamento implementado
R01	Uma mídia só pode entrar em uma locação se estiver disponível.	Locacao	adicionarMidia()	Testa isDisponivel(); se falso, não adiciona e imprime mensagem.
R02	Mídia alugada fica indisponível.	Locacao	adicionarMidia()	Chama setDisponivel(false) após adicionar.
R03	Na devolução, todas as mídias voltam ao acervo.	Locacao	finalizarDevolucao()	Percorre as mídias e chama setDisponivel(true).
R04	Atraso gera multa de R\$ 5,00 por dia.	Locacao e Cliente	finalizarDevolucao(), adicionarMulta()	Se dias > 0, calcula dias × 5,00 e soma ao saldo devedor.
R05	Só aluga quem está ativo e sem multa pendente.	Cliente	podeAlugar()	Retorna ativo && saldoDevedor == 0.0; a verificação é feita no index.php.
R06	DVD arranhado tem 10% de desconto.	DVD	calcularPrecoLocacao()	Multiplica o total por 0,90 quando arranhado é verdadeiro.
R07	Blu-Ray 3D tem taxa fixa de R\$ 5,00.	BluRay	calcularPrecoLocacao()	Soma 5,00 ao total quando is3D é verdadeiro.
R08	Cliente novo começa ativo e sem dívidas; mídia nova começa disponível.	Cliente, Midia	__construct()	Valores iniciais definidos no construtor.
R09	Não é possível criar uma pessoa ou mídia genérica.	Pessoa, Midia	—	Classes abstract; o PHP bloqueia a instanciação direta.

12. Dependências entre arquivos/classes

```

index.php
├── Locadora.php
│   ├── Cliente.php — Pessoa.php
│   ├── Funcionario.php — Pessoa.php
│   ├── Midia.php
│   └── Locacao.php — Cliente.php, Funcionario.php, Midia.php
├── Cliente.php
├── Funcionario.php
├── DVD.php — Midia.php
└── BluRay.php — Midia.php
  
```

Origem	Dependência	Motivo
index.php	Locadora, Cliente, Funcionario, DVD, BluRay (require_once)	Carrega as classes necessárias para a simulação.
Locadora.php	Cliente, Funcionario, Midia, Locacao (require_once)	Usa esses tipos nos parâmetros e nas coleções.
Locacao.php	Cliente, Funcionario, Midia (require_once)	Usa esses tipos nos atributos e parâmetros.
Cliente.php, Funcionario.php	Pessoa.php (require_once e extends)	Herança da superclasse.
DVD.php, BluRay.php	Midia.php (require_once e extends)	Herança da superclasse.

13. Casos de teste

Os testes foram executados pelo `index.php`, que simula o fluxo completo do sistema (ambiente XAMPP/VS Code e linha de comando).

ID	Cenário/entrada	Operação	Resultado esperado	Resultado obtido	Situação
T01	Cliente novo, recém-criado.	<code>podeAlugar()</code>	<code>true</code> : ativo e sem dívidas.	Mensagem “Cliente ... está apto para alugar.”	APROVADO
T02	Acervo com 2 DVDs e 1 Blu-Ray cadastrados.	<code>listarAcervoDisponivel()</code>	3 linhas, com tipo e preço base.	[DVD] R\$ 5,00, [DVD] R\$ 5,00 e [BluRay] R\$ 8,00.	APROVADO
T03	Locação de 3 dias com as 3 mídias.	<code>adicionarMidia() × 3</code> e <code>listarAcervoDisponivel()</code>	Mídias indisponíveis; acervo sem itens.	Somente o cabeçalho do acervo é exibido.	APROVADO
T04	3 dias: DVD comum, DVD arranhado e Blu-Ray 3D.	<code>calcularValorTotal()</code>	15,00 + 13,50 + 29,00 = 57,50.	“R\$ 57,50”.	APROVADO
T05	Objetos Cliente e Funcionario.	<code>getNome()</code> herdado de Pessoa	Nomes exibidos corretamente.	“Atendido por: Carlos Oliveira” e “Cliente: João Victor Fagundes”.	APROVADO
T06	Devolução com 2 dias de atraso.	<code>finalizarDevolucao(2)</code>	Saldo devedor = 2 × 5,00 = 10,00.	“R\$ 10,00”.	APROVADO
T07	Cliente com saldo devedor de R\$ 10,00.	<code>podeAlugar()</code>	<code>false</code> .	“NÃO (Multa Pendente)”.	APROVADO
T08	Após a devolução.	<code>listarAcervoDisponivel()</code>	As 3 mídias voltam ao acervo.	As 3 linhas são exibidas novamente.	APROVADO

Situação: APROVADO / REPROVADO / CORRIGIDO. Todos os casos acima foram aprovados.

14. Histórico técnico de versões

Versão	Data	Conteúdos incorporados	Principais alterações
1.0	29/09/2026	Classes e objetos, encapsulamento, construtores, getters e setters, associação, agregação, herança, sobrescrita, polimorfismo e Git/GitHub.	Primeira versão completa publicada no GitHub (commit 6e1791d): 8 classes, <code>index.php</code> e <code>README.md</code> .
1.0	30/09/2026	Documentação técnica.	Inclusão da documentação técnica em DOCS/ (commit f4fccfb), sem alterar o código-fonte.

15. Limitações técnicas conhecidas

- **Persistência:** os dados ficam apenas em memória (arrays) e desaparecem quando o script termina.
- **Interface:** a saída é em texto, pelo `index.php`; não há interface gráfica nem entrada de dados pelo usuário.
- **Dias de atraso:** são informados manualmente em `finalizarDevolucao()`. O atributo `dataLocacao` é gravado, mas não é usado para calcular o atraso.

- **Validações:** os setters e `adicionarMulta()` não validam valores (por exemplo, multa negativa, nome vazio, CPF ou e-mail inválidos).
- **Regra do cliente apto:** `Locacao` não consulta `podeAlugar()`; a verificação é feita por quem cria a locação (no `index.php`).
- **Quitação de multa:** não existe método para pagar o saldo devedor; o cliente com multa continua impedido de alugar.
- **Escopo do teste:** o `index.php` é um roteiro de demonstração sem framework de testes. A tentativa de alugar uma mídia já alugada está implementada, mas não é exercitada por ele.
- **Valor da multa:** a taxa de R\$ 5,00 por dia está escrita diretamente no código de `Locacao`.
- **Caminhos:** o `index.php` carrega `/classes/` com letra minúscula, e a pasta se chama `Classes`. Isso funciona em Windows/XAMPP, mas pode exigir ajuste em sistemas que diferenciam maiúsculas de minúsculas.

16. Possíveis evoluções técnicas

- **Persistência em banco de dados** (MySQL/MariaDB com PDO), substituindo os arrays em memória.
- **Cálculo automático do atraso** a partir de `dataLocacao` e da data de devolução.
- **Validações nos métodos:** valores positivos em `adicionarMulta()`, dados válidos nos setters e checagem de `podeAlugar()` dentro de `Locacao`.
- **Pagamento de multas:** método para quitar o saldo devedor, integrado a um gateway (PIX ou cartão).
- **Interface web responsiva** para atendentes e consulta do acervo pelos clientes.
- **Reservas antecipadas** de mídias muito procuradas.
- **Novas categorias de mídia** (4K Ultra HD, jogos, edições de colecionador) como novas subclasses de `Midia`, sem alterar `Locacao`.
- **Testes automatizados** (por exemplo, PHPUnit) e carregamento automático de classes (autoload).

17. Evidências da versão final

URL do repositório	https://github.com/Vyctorr7/Locadora
Branch/versão final	main — versão 1.0
Commit final (hash ou identificação)	6e1791d (29/09/2026), commit do código-fonte. Os commits seguintes alteram apenas a documentação.
Data da validação	30/09/2026

18. Declaração de revisão da documentação

Conferência realizada antes da entrega, confrontando a documentação com o código-fonte da versão final.

- ☒ O diagrama corresponde às classes implementadas.
- ☒ Os atributos e métodos documentados existem no código.
- ☒ Os relacionamentos descritos podem ser identificados na implementação.
- ☒ Os conceitos de POO citados foram efetivamente aplicados.
- ☒ Os testes registrados foram executados.
- ☒ O README.md e esta documentação apontam para a mesma versão do projeto.
- ☒ Conteúdos produzidos com apoio de IA foram revisados e validados.